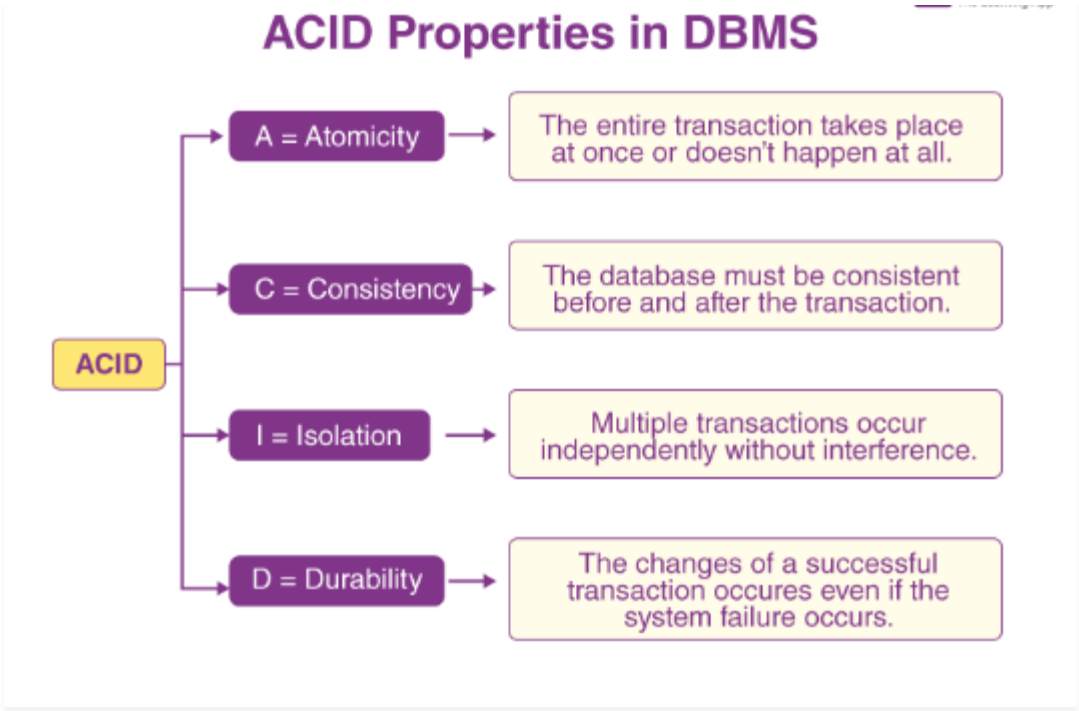


A – Atomicity

Atomicity means that an entire transaction either takes place all at once or it doesn't occur at all. It means that there's no midway. The transactions can never occur partially. Every transaction can be considered as a single unit, and they either run to completion or do not get executed at all. We have the following two operations here:

—Commit: In case a transaction commits, the changes made are visible to us. Thus, atomicity is also called the 'All or nothing rule'.

—Abort: In case a transaction aborts, the changes made to the database are not visible to us.

Consider this transaction T that consists of T1 and T2: Transfering 100 from account A to account B.

Before: A : 500	B : 200
Transaction T	
T1	T2
Read (A) A: = A - 100 Write (A)	Read (B) B: = B + 100 Write (B)
After: A : 400	B : 300

In case the transaction fails when the T1 is completed but the T2 is not completed (say, after write(A) but before write(B)), then the amount has been deducted from A but not added to B. This would result in a database state that is inconsistent. Thus, the transaction has to be executed in its entirety in order to ensure the correctness of the database state.

C – Consistency

Consistency means that we have to maintain the integrity constraints so that any given database stays consistent both before and after a transaction. If we refer to the example discussed above, then we have to maintain the total amount, both before and after the transaction.

Total after T occurs = 400 + 300 = 700.

Total before T occurs = 500 + 200 = 700.

Thus, the given database is consistent. Here, an inconsistency would occur when T1 completes, but then the T2 fails. As a result, the T would remain incomplete.

I – Isolation

Isolation ensures the occurrence of multiple transactions concurrently without a database state leading to a state of inconsistency. A transaction occurs independently, i.e. without any interference. Any changes that occur in any particular transaction would NOT be ever visible to the other transactions unless and until this particular change in this transaction has been committed or written to the memory.

The property of isolation ensures that when we execute the transactions concurrently, it will result in such a state that's equivalent to the achieved state that was serially executed in a particular order.

Let A = 500, B = 500

Let us consider two transactions here- T and T"

T	T''
Read (A) A: = A * 100 Write (A) Read (B) B: = B - 50 Write (B)	Read (A) Read (B) Z: = A + B Write (Z)

Suppose that T has been executed here till Read(B) and then T'' starts. As a result, the interleaving of operations would take place. And due to this, T'' reads the correct value of A but incorrect value of B.

T'': (X+B = 50, 000+500=50, 500)

Thus, the sum computed here is not consistent with the sum that is obtained at the end of the transaction:

T: (A+B = 50, 000 + 450 = 50, 450).

It results in the inconsistency of a database due to the loss of a total of 50 units. The transactions must, thus, take place in isolation. Also, the changes must only be visible after we have made them on the main memory.

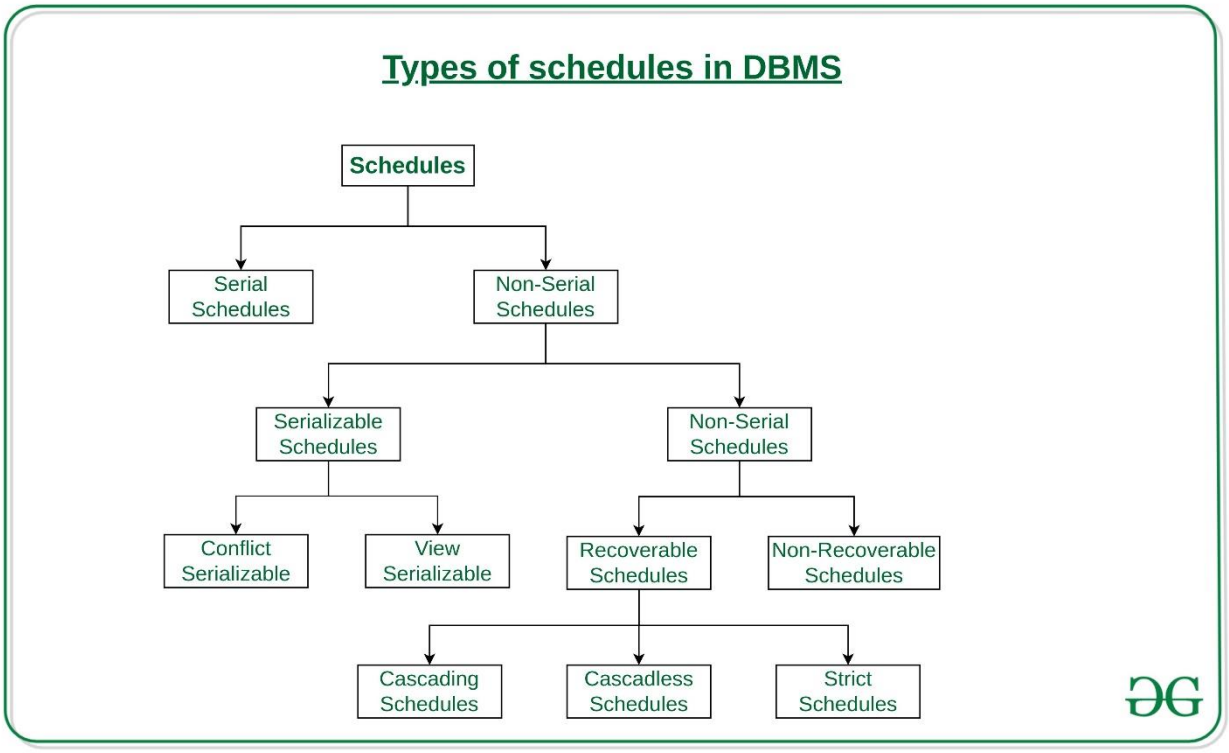
D – Durability

The durability property states that once the execution of a transaction is completed, the modifications and updates on the database gets written on and stored in the disk. These persist even after the occurrence of a system failure. Such updates become permanent and get stored in non-volatile memory. Thus, the effects of this transaction are never lost.

Uses of ACID Properties

In totality, the ACID properties of transactions provide a mechanism in DBMS to ensure the consistency and correctness of any database. It ensures consistency in a way that every transaction acts as a group of operations acting as single units, produces consistent results, operates in an isolated manner from all the other operations, and makes durably stored updates. These ensure the integrity of data in any given database.

SCHEDULES



What is Serializability in DBMS?

Serializability in DBMS ensures safe parallel execution of transactions by enforcing rules, preventing unexpected or incorrect results. Each transaction is treated as if it's the only one running, maintaining integrity.

DBMS employs a technique called **concurrency control** to achieve serializability. This technique helps ensure that multiple transactions do not interfere with each other's execution by controlling their access to shared resources. These shared resources can be database tables and records. By maintaining serializability, the DBMS ensures the correctness and consistency of the database even under heavy concurrent usage.

Types of Schedules in DBMS

There are two types of schedules in DBMS:

Serial schedule

These are those schedules in which transactions are executed sequentially, meaning a new transaction doesn't begin until the previous one has concluded.

Example

Transaction	Operation
T1	Read(X)
T1	Write(X)
T2	Read(Y)
T2	Write(Y)

Non-serial schedule

This scheduling method involves interleaving the operations of multiple transactions, potentially increasing concurrency challenges. Transactions are executed concurrently, yet the final outcome remains consistent and identical to a serial schedule. It is further divided into two parts:

1. Serializable(Conflict and View Serializable)
2. Non- serializable(Recoverable and Non-recoverable)

Example

Transaction	Operation
T1	Read(X)
T2	Read(Y)
T1	Write(X)
T2	Wri(Y)

Types of Serializability in DBMS

In DBMS (Database Management System), there are two types of serializability:

1. **Conflict Serializability**
2. **View Serializability**

Conflict Serializability

Conflict serializability is a property of a schedule. It ensures that it is equivalent to a serial schedule. A schedule can be declared as conflict serializable if and only if all conflicting pairs of transactions in the schedule have the same relative order in all serial schedules that are equivalent to the given schedule.

A **conflicting pair of transactions** is a pair of transactions that access the same data item, and at least one of them is a write operation. This type of serializability can be determined by drawing a precedence graph of the transactions in the schedule and checking if the graph is acyclic. If the graph is acyclic, then the schedule is conflict serializable. If it contains a cycle, then the schedule is not conflict serializable.

Let us understand this with an example.

Example

Consider a schedule with three transactions, T1, T2, and T3, and the following set of operations:

```
T1: R(P), W(Q)
T2: R(Q), W(R)
T3: R(R), W(P)
```

The operations that are performed in the transactions are R(Read) and W(Write) operations on the data items 'P' and 'Q'.

In a precedence graph, the edges represent the ordering constraints between transactions in the concurrent execution of a database system. If we encounter any conflicts between transactions, then we will draw an edge. The conflict serializability of the schedule can be determined by constructing a precedence graph as follows:

```
T1 -> T2
T2 -> T3
T3 -> T1
```

View Serializability

View serializability is a property of a schedule. It ensures that it is equivalent to a serial schedule with the same set of read and write operations. A schedule is declared as view serializable if and only if it produces the same set of values for all read operations as any serial schedule that is equivalent to the given schedule.

View serializability can be determined by constructing a view serializable schedule graph (VSG) of the transactions in the schedule. The VSG contains nodes representing the transactions and edges representing conflicts and views between the transactions. If the VSG is acyclic, then the schedule is view serializable.

Let us understand this with an example.

Example

Consider a schedule with three transactions, T1, T2, and T3, and the following set of operations:

```
T1: R(P), W(Q)
T2: R(Q), W(R)
T3: R(R), W(P)
```

The operations that are performed in the transactions are R(Read) and W(Write) operations on the data items 'P' and 'Q'. In a VSG, the edges represent the ordering constraints between transactions in the concurrent execution of a database system. It ensures that the transactions are view serializable.

We must first identify the view equivalent schedules for the given concurrent execution to obtain the edges in this graph. Two schedules are view equivalent if they produce the same set of database states for any initial database state. The view serializability of the schedule can be determined by constructing a VSG as follows:

```
T1 -> T2
T2 -> T3
T3 -> T1
```

Recoverable schedule

Consider the following example –

T1	T2
R(X)	
W(X)	
	W(X)
	R(X)
READ1(B)	
commit	
	commit

Here, transaction T2 is reading the value written by transaction T1 and the commit of T2 occurs after the commit of T1. Hence, it is a recoverable schedule.

Again the **recoverable schedule is divided into cascade less and strict schedule** –

- **Cascadeless schedule**

Given below is an example for the cascadeless schedule –

T1	T2
R(X)	
W(X)	
	W(X)
commit	
	R(X)
	commit

Here, the updated value of **X** is read by transaction T2 only after the commit of transaction T1. Hence, the schedule is **Cascadeless schedule**.

- **Strict schedule**

Given below is an example of strict schedule –

T1	T2
R(X)	
	R(X)
W(X)	
commit	
	W(X)
	R(X)
	commit

Here, transaction T2 reads and writes the updated or written value of transaction T1 only after the transaction T1 commits. Hence, the schedule is **strict schedule**.

Now let us see about the Non-recoverable schedule.

Non-Recoverable Schedule

A schedule that is not recoverable is non-recoverable. If the commit operation of Ti doesn't occur before the commit operation of Tj, it is non-recoverable.

Consider an example for the non-recoverable schedule as given below –

Schedule 1

T1	T2
read(x)	
x=x-n	
write(x)	
	read(x)
	x=x+n
	write(x)
	commit

CONCURRENCY

Concurrency Control Problems

Different problems may arise when the concurrent execution of transactions is being done in an environment. When proper concurrent control methods are not used, it may lead to data inconsistency in the database. Below are some of the common problems in concurrency control:

1. Phantom Read Problem

A phantom read problem is also known as a dirty-read problem. It is a common problem that occurs when a transaction tries to read the set of rows that satisfy a specific condition. But at the same time, another transaction occurs to perform some operation on the rows that affect the result set before the first transaction completes. As a result, there is a difference or inconsistency in the output of the first transaction when it re-executes the same query.

In simpler words, A dirty read problem occurs when one transaction updates a database item, and then the transaction fails for some reason. The updated database item exists by another transaction before changing back to the original value.

For example, consider a schedule S in which transaction T2 rollbacks after the read (A, a) operation of transaction T4. Now, T3 and T4 have the read values of a which were never committed. So, they perform the dirty read operation. This is so because transactions T3 and T4 read the value of a modified transaction T2. Since the transaction T2 fails and rollback it means that T2 obtains the original value of A and cancels the modified value of A. However, other transactions like T3 and T4 process the modified value of a and result in an inconsistent date in the database.

Schedule (S):

Lock – X (A)
Read (A, a)
Lock – S(B)
Read (B, b)
Write (A, a)
Unlock (A) Lock – X (A)
Read (A, a)
Write (A, a)
Unlock (A) Lock – S(A)
Read (A, a)

2. Lost Update Problem

A lost Update problem occurs when two transactions that exceed the same data items have their operations in a way that makes the value of some database item incorrect. This occurs when two or more transactions read the same data and then update it based on the real value. Since each transaction does not see the updates made by the other transactions, the final value reflects only the last update, effectively “losing” the previous updates.

For example, if two transactions read the same account balance and then deduct an amount without properly locking, one of the subtractions may be lost. Consider another example, where a bank account has 5000 as a balance. There are two transactions T1 and T2, first, the transaction T1 reads the balance and gets the result as 5000. Now T2 reads the balance from the account and gets the result as 5000.

T1 now updates the balance by subtracting 2000 from the initial state of account balance. While T2 updates the balance after subtracting 1000. Transaction T2 reads the initial balance of 5000 and updates it to 4000, effectively ignoring the update made by Transaction T1. But we expected the final state as 5000 – 2000 – 1000 = 2000. As a result, the update by T1 is lost.

T2	T3	T4
rollback		

Techniques for Concurrency Control

Lock-based Protocols

Lock-based protocols use locks to control access to data items. A lock is a mechanism to control concurrent access to a data item. Transactions acquire locks before accessing data and release them after completing their operations.

Types of Locks
1. **Shared Lock (S):** Allows multiple transactions to read a data item but not modify it.
2. **Exclusive Lock (X):** Allows a single transaction to read and modify a data item.

Two-Phase Locking (2PL)
Two-phase locking is a protocol that ensures serializability. It operates in two phases:

1. **Growing Phase:** A transaction may acquire locks but not release any.
2. **Shrinking Phase:** A transaction may release locks but not acquire any new ones.

Timestamp-based Protocols

Timestamp-based protocols assign a unique timestamp to each transaction. Transactions are ordered based on their timestamps, ensuring serializability.

Basic Timestamp Ordering
Each transaction is assigned a unique timestamp. The protocol ensures that conflicting operations are executed in timestamp order. If a transaction tries to perform an operation that violates the order, it is rolled back and restarted with a new timestamp.

Thomas's Write Rule
An optimization to basic timestamp ordering, Thomas's Write Rule allows certain operations to be ignored if they do not affect the outcome, reducing unnecessary rollbacks.

Optimistic Concurrency Control

Optimistic concurrency control expects that conflicts are rare. Before being checked for conflicts at the commit point, transactions can proceed without limitations.

Phases of Optimistic Concurrency Control
1. **Read Phase:** Transactions execute without restrictions and read data items.
2. **Validation Phase:** Transactions are checked for conflicts before being committed.
3. **Write Phase:** If no conflicts are found, the transactions are committed; otherwise, they are rolled back.

Remediation Steps for Concurrency Control

The term "remediation steps" refers to the actions or procedures that can be taken to resolve the concurrency control issues that occur in database transactions. These steps provide the benefit of multiple concurrent transactions accessing the same data that don't interact with one another and cause inconsistent data in the DBMS. For addressing the concurrency control, there are some remediation steps:

- **Proper Locking:** Applying locking mechanisms like shared lock or exclusive lock can help avoid lost updates and unclean reads by the users.
- **Isolation levels:** To determine the degree of performance and consistency of the data, isolation levels are used. Isolation levels include Read Uncommitted, Read Committed, Serializable, etc.
- **Deadlock Handling**: A deadlock is a situation that occurs when multiple transactions wait for an infinite period for one another to release resources. Use deadlock detection and resolution mechanisms to efficiently manage deadlocks.
- **Transaction Management:** Implement transaction management strategies to assure atomicity and durability.

Advantages of Concurrency

Concurrency means the execution of multiple transitions concurrently, which can significantly better utilise resources, increase throughput, improve user experience, and bring several benefits. Some of the advantages of concurrency are:

- Better utilisation of resources like CPU, main memory, etc. When concurrent transactions are executed, the DBMS ensures that the processors are not idle, and utilising every resource of the system leads to more efficient operation.
- Transactions that use concurrency don't have to wait for other transactions to finish before beginning. Users may see quicker response times as a result of this decreasing the total transaction waiting time.

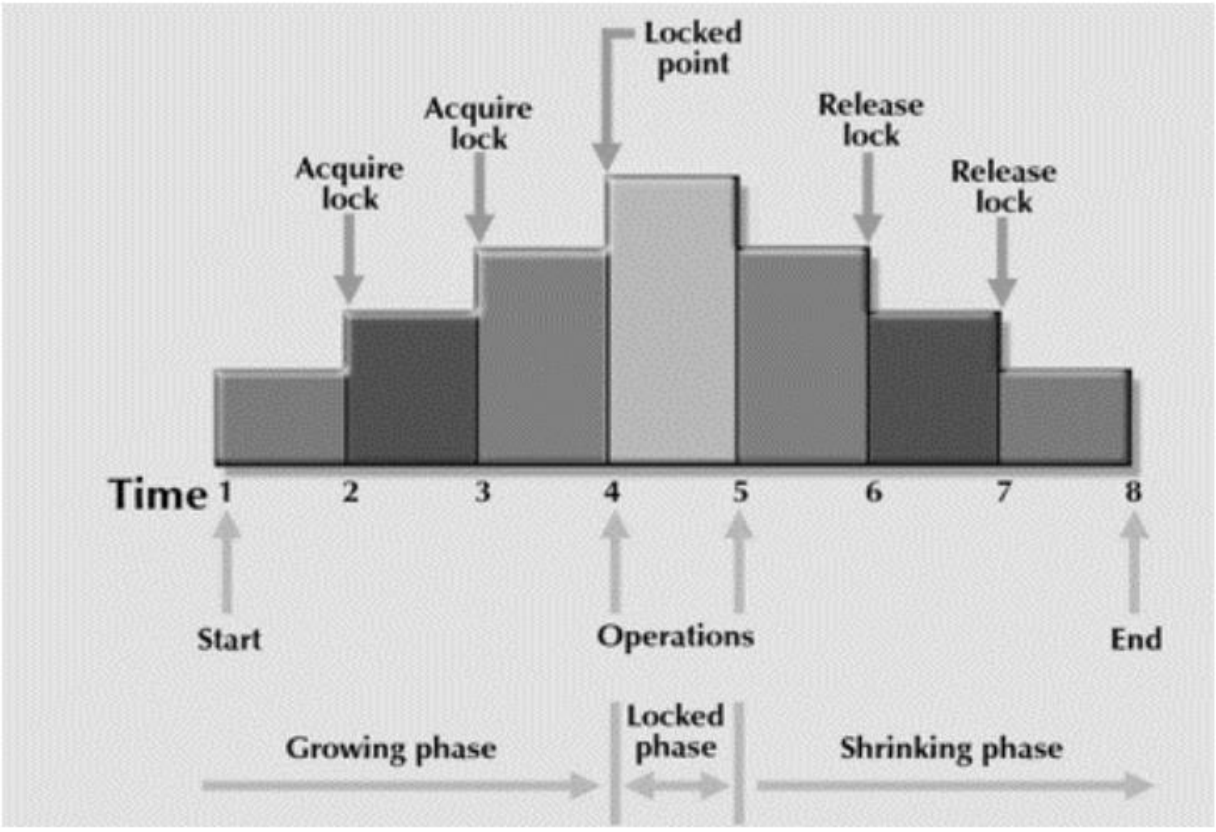
Two Phase Locking Protocol

2PL locking protocol

Every **transaction** will lock and unlock the data item in two different phases.

- **Growing Phase** – All the locks are issued in this phase. No locks are released, after all changes to data-items are committed and then the second phase (shrinking phase) starts.
- **Shrinking phase** – No locks are issued in this phase, all the changes to data-items are noted (stored) and then locks are released.

The 2PL locking protocol is represented diagrammatically as follows –



In the growing phase transaction reaches a point where all the locks it may need has been acquired. This point is called LOCK POINT.

Isolation Levels

1. **Read Uncommitted**: The lowest level, where transactions can see uncommitted changes made by other transactions.
2. **Read Committed**: Transactions can only see changes that have been committed while the transaction is running. However, if the same data is read again within the same transaction, it's possible to see different values (non-repeatable reads and phantom reads)
3. **Repeatable Read**: Ensures that if a transaction reads a record, it will read the same value every time, regardless of changes made by other transactions. However, it is still possible to have phantom reads (new records inserted by other transactions can be seen if they match the criteria).
4. **Serializable**: The highest level, which ensures full serializability of transactions. It is as if transactions are processed one after another rather than concurrently.

Each level is a trade-off between consistency and performance: higher isolation levels provide greater consistency but typically at the cost of concurrency.

Databases use a variety of methods to manage concurrency, including locking systems such as **Two phase locking (2PL)**, **Multiversion concurrency Control (MVCC)**, and other techniques. These are all designed to ensure data integrity while maximizing the database's ability to handle multiple requests at the same time.

Two Phase Locking

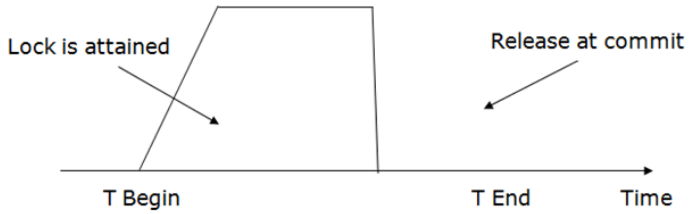
Two-phase locking (2PL) is a concurrency control method used in database systems to ensure **transaction serializability**. Transaction serializability ensures that even when database transactions overlap in time, the final result looks if they had occurred one after another in sequence. The protocol works in two phases:

1. **Locking (Growing) Phase**: In this phase, a transaction may obtain locks on data (rows, tables etc) but cannot release any. There are mainly two types of locks:
   - **Shared Lock**: Allows a transaction to read a resource. Multiple transactions can hold shared locks on the same resource, which means multiple readers can read the same data.
   - **Exclusive Lock**: Allows a transaction to both read and write a resource. If a transaction holds an exclusive lock on a resource, no other transaction can acquire any lock (shared or exclusive) on that resource.
2. **Unlocking (Shrinking) Phase:** The transaction releases all the locks in this phase but cannot acquire any new ones. locks can be released as soon as the transaction no longer needs the locked resource

In 2PL, multiple readers can access data simultaneously through shared locks. However, this system struggles with concurrent reads and writes: reader and writer transactions interfere with each other since readers' shared locks are incompatible with writers' exclusive locks.
The lock compatibility would be like this:

4. Strict Two-phase locking (Strict-2PL)

- The first phase of Strict-2PL is similar to 2PL. In the first phase, after acquiring all the locks, the transaction continues to execute normally.
- The only difference between 2PL and strict 2PL is that Strict-2PL does not release a lock after using it.
- Strict-2PL waits until the whole transaction to commit, and then it releases all the locks at a time.
- Strict-2PL protocol does not have shrinking phase of lock release.



It does not have cascading abort as 2PL does.

Timestamp Ordering Protocol

- The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps. The order of transaction is nothing but the ascending order of the transaction creation.
- The priority of the older transaction is higher that's why it executes first. To determine the timestamp of the transaction, this protocol uses system time or logical counter.
- The lock-based protocol is used to manage the order between conflicting pairs among transactions at the execution time. But Timestamp based protocols start working as soon as a transaction is created.
- Let's assume there are two transactions T1 and T2. Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times. T1 has the higher priority, so it executes first as it is entered the system first.
- The timestamp ordering protocol also maintains the timestamp of last 'read' and 'write' operation on a data.

Basic Timestamp ordering protocol works as follows:

1. Check the following condition whenever a transaction Ti issues a **Read (X)** operation:

- If W_TS(X) >TS(Ti) then the operation is rejected.
- If W_TS(X) <= TS(Ti) then the operation is executed.
- Timestamps of all the data items are updated.

2. Check the following condition whenever a transaction Ti issues a **Write(X)** operation:

- If TS(Ti) < R_TS(X) then the operation is rejected.
- If TS(Ti) < W_TS(X) then the operation is rejected and Ti is rolled back otherwise the operation is executed.

Validation Based Protocol

Validation phase is also known as optimistic concurrency control technique. In the validation based protocol, the transaction is executed in the following three phases:

1. **Read phase:** In this phase, the transaction T is read and executed. It is used to read the value of various data items and stores them in temporary local variables. It can perform all the write operations on temporary variables without an update to the actual database.
2. **Validation phase:** In this phase, the temporary variable value will be validated against the actual data to see if it violates the serializability.
3. **Write phase:** If the validation of the transaction is validated, then the temporary results are written to the database or system otherwise the transaction is rolled back.

Here each phase has the following different timestamps:

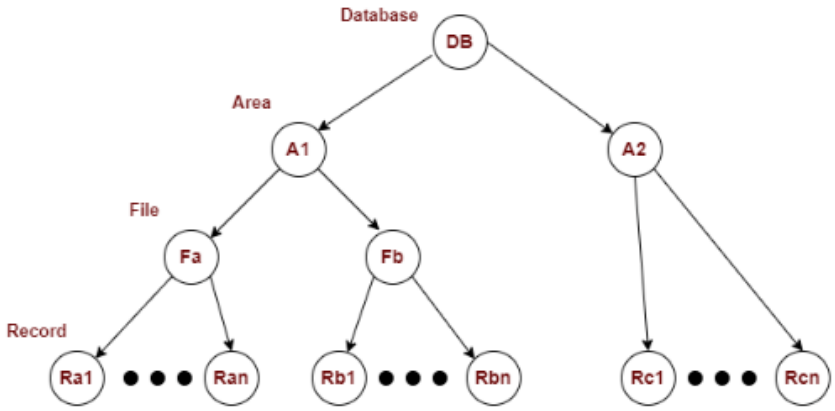
Start(Ti): It contains the time when Ti started its execution.

Validation (T_i): It contains the time when Ti finishes its read phase and starts its validation phase.

Finish(Ti): It contains the time when Ti finishes its write phase.

- This protocol is used to determine the time stamp for the transaction for serialization using the time stamp of the validation phase, as it is the actual phase which determines if the transaction will commit or rollback.
- Hence TS(T) = validation(T).
- The serializability is determined during the validation process. It can't be decided in advance.
- While executing the transaction, it ensures a greater degree of concurrency and also less number of conflicts.
- Thus it contains transactions which have less number of rollbacks.

Multiple



Multi Granularity tree Hierarchy

Granularity

Shared Lock

It allows multiple transactions to read the same data simultaneously. It is used to prevent other transactions from modifying the data while a transaction is reading it.

Exclusive Lock

It prevents any other transaction from accessing the data. It is used to prevent other transactions from reading or modifying the data while a transaction is writing to it.

Different Types of Intention Mode Locks in Multiple Granularity

Intention mode locks are a type of lock used in multiple granularity locking that allows multiple transactions to acquire locks on the same resource, but with different levels of access.

There are three types of intention mode locks in multiple granularity locking –

Intent Shared (IS) Locks

This lock is used when a transaction needs to read a resource but does not intend to modify it. It indicates that the transaction wants to acquire a Shared lock on a resource.

Intent Exclusive (IX) Locks

This lock is used when a transaction needs to modify a resource but does not intend to share it. It indicates that the transaction wants to acquire an Exclusive lock on a resource.

Shared with Intent Exclusive (SIX) Locks

This lock is used when a transaction intends to acquire both Shared and Exclusive locks on a resource. It indicates that the transaction wants to acquire an Exclusive lock on a resource after acquiring Shared locks on other resources.

These intention mode locks are used to optimize the locking mechanism in a database by allowing transactions to acquire locks on multiple resources in a coordinated manner. They help prevent deadlocks and improve concurrency in a database system.

The compatibility matrix for these lock modes are described below:

Compatibility Matrix

	IS	IX	S	SIX	X
IS	YES	YES	YES	YES	NO
IX	YES	YES	NO	NO	NO
S	YES	NO	YES	NO	NO
SIX	YES	NO	NO	NO	NO
X	NO	NO	NO	NO	NO

Intention lock modes are utilized in the multiple-granularity locking protocol to ensure serializability. According to this protocol, when a transaction (T) attempts to lock a node, it must adhere to the following guidelines

- Transaction T must follow the lock-compatibility matrix.
- Transaction T must initially lock the root of the tree in any mode.
- Transaction T may only lock a node in S or IS mode if it has already locked the node's parent in either IS or IX mode.
- Transaction T may only lock a node in IX, SIX, or X mode if it has already locked the parent of the node in either SIX or IX modes.
- Transaction T may only lock a node if it has not yet unlocked any nodes (i.e., it is two- phase).
- Transaction T may only unlock a node if it is not currently holding any locks on its child nodes.

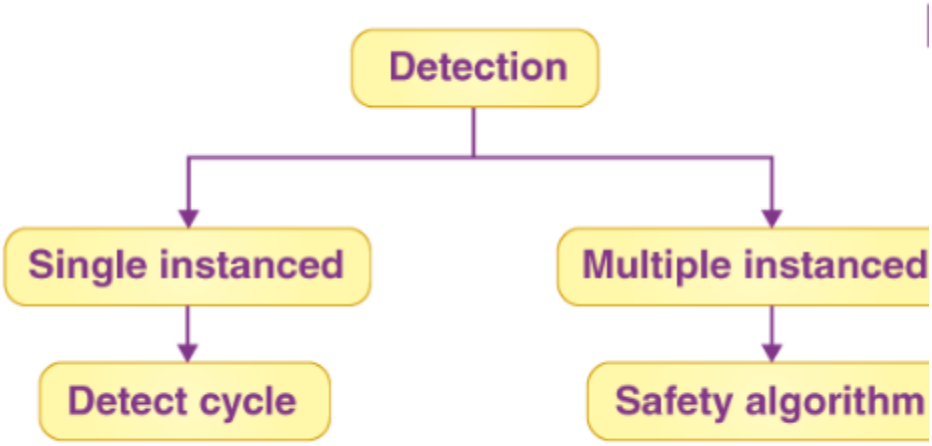
In the multiple granularity protocol, locks must be acquired in a top-down (root-to-leaf) manner, while they must be released in a bottom-up (leaf-to-root) manner. This ensures that transactions can access shared resources efficiently while avoiding deadlocks.

Deadlock Handling

How to Detect Deadlock in Operating Systems?

The OS does not use any mechanisms to avoid or prevent deadlocks in this approach. As a result, the system predicts that the deadlock will occur. The OS periodically scans the system for any deadlocks in order to avoid them. If any deadlocks are discovered, the OS will attempt to restore the system using several ways.

The OS's primary responsibility is to detect deadlocks. With the help of the resource allocation graph, the OS can detect deadlocks.



If a cycle forms in a system with single instanced resource types, there will undoubtedly be a deadlock. Detecting a cycle, on the other hand, is insufficient in a graph of the multiple instanced resource type. By turning the resource allocation graph into the allocation matrix as well as the request matrix, we must apply the safety algorithm to the system.

Deadlock Recovery in Operating System

OS examines either resources or processes to recover the system from deadlocks.

For Resource

Preempt the resource

We can take one of the resources from the resource owner (process) and give it to another process in the hopes that it will finish the execution and release the resource sooner. Choosing a resource that will be snatched will be challenging.

Rollback to a safe state

To reach the deadlock state, the system happens to go through several states. The operating system has the ability to restore the system to a previous safe state. The OS must implement checkpointing at each state for this to work.

When we reach a deadlock, we must reverse all allocations and return to the prior safe state.

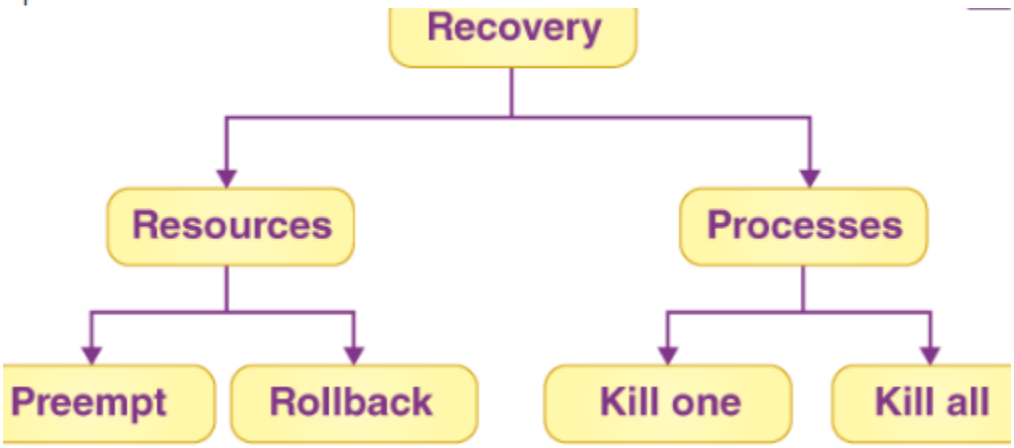
For Process

Kill a process

Our problem can be solved by killing a process; however, the bigger issue is deciding which process to kill. A process that has done the least amount of work till now is usually killed by the operating system.

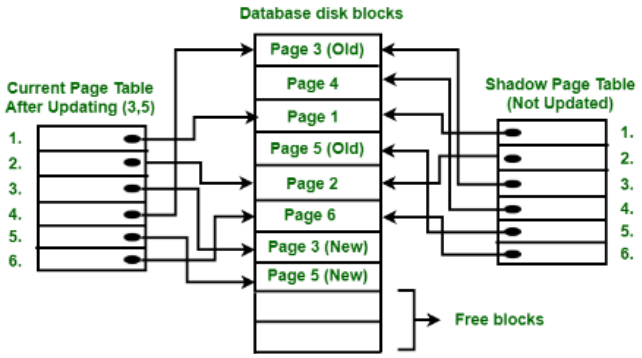
Kill all processes

This is not a persuasive strategy, but it may be used if the problem becomes extremely serious. Killing all processes will result in system inefficiencies because all processes will have to start over.



Shadow Paging

Shadow Paging is recovery technique that is used to recover database. In this recovery technique, database is considered as made up of fixed size of logical units of storage which are referred as **pages.** pages are mapped into physical blocks of storage, with help of the **page table** which allow one entry for each logical page of database. This method uses two page tables named **current page table** and **shadow page table**. The entries which are present in current page table are used to point to most recent database pages on disk. Another table i.e., Shadow page table is used when the transaction starts which is copying current page table. After this, shadow page table gets saved on disk and current page table is going to be used for transaction. Entries present in current page table may be changed during execution but in shadow page table it never get changed. After transaction, both tables become identical. This technique is also known as **Cut-of-Place updating.**



To understand concept, consider above figure. In this 2 write operations are performed on page 3 and 5. Before start of write operation on page 3, current page table points to old page 3. When write operation starts following steps are performed :

1. Firstly, search start for available free block in disk blocks.
2. After finding free block, it copies page 3 to free block which is represented by Page 3 (New).
3. Now current page table points to Page 3 (New) on disk but shadow page table points to old page 3 because it is not modified.
4. The changes are now propagated to Page 3 (New) which is pointed by current page table.

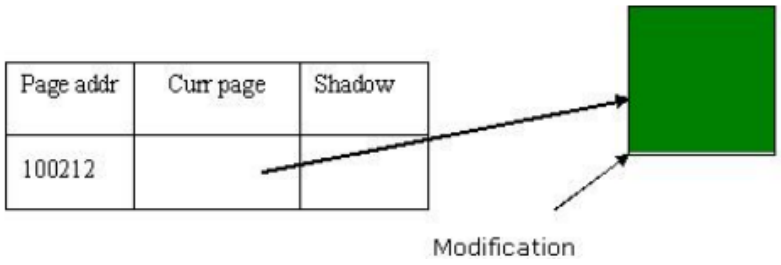
COMMIT Operation : To commit transaction following steps should be done :

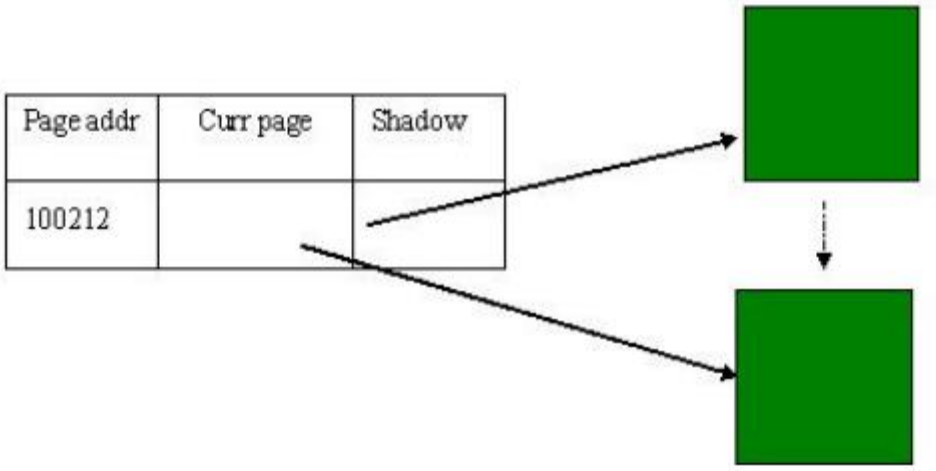
1. All the modifications which are done by transaction which are present in buffers are transferred to physical database.
2. Output current page table to disk.
3. Disk address of current page table output to fixed location which is in stable storage containing address of shadow page table. This operation overwrites address of old shadow page table. With this current page table becomes same as shadow page table and transaction is committed.

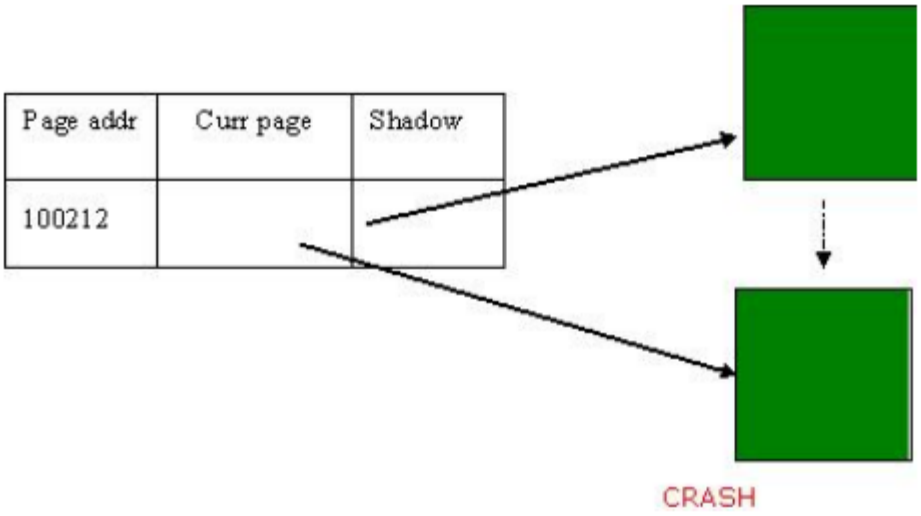
Failure : If system crashes during execution of transaction but before commit operation, With this, it is sufficient only to free modified database pages and discard current page table. Before execution of transaction, state of database get recovered by reinstalling shadow page table. If the crash of system occur after last write operation then it does not affect propagation of changes that are made by transaction. These changes are preserved and there is no need to perform redo operation. **Advantages :**

Shadow Paging

Shadow paging is an alternative to log-based recovery techniques, which has both advantages and disadvantages. It may require fewer disk accesses, but it is hard to extend paging to allow multiple concurrent transactions. The paging is very similar to paging schemes used by the operating system for memory management.
The idea is to maintain two page tables during the life of a transaction: the *current* page table and the *shadow* page table. When the transaction starts, both tables are identical. The shadow page is never changed during the life of the transaction. The current page is updated with each **write**operation. Each table entry points to a page on the disk. When the transaction is committed, the shadow page entry becomes a copy of the current page table entry and the disk block with the old data is released. If the shadow is stored in nonvolatile memory and a system crash occurs, then the shadow page table is copied to the current page table. This guarantees that the shadow page table will point to the database pages corresponding to the state of the database prior to any transaction that was active at the time of the crash, making aborts automatic.




Page addr
Curr page
Shadow
100212


Page addr
Curr page
Shadow
100212
CRASH

There are drawbacks to the shadow-page technique:

1. **Commit overhead.** The commit of a single transaction using shadow paging requires multiple blocks to be output -- the current page table, the actual data and the disk address of the current page table. Log-based schemes need to output only the log records.
2. **Data fragmentation.** Shadow paging causes database pages to change locations (therefore, no longer contiguous.
3. **Garbage collection.** Each time that a transaction commits, the database pages containing the old version of data changed by the transactions must become inaccessible. Such pages are considered to be *garbage* since they are not part of the free space and do not contain any usable information. Periodically it is necessary to find all of the garbage pages and add them to the list of free pages. This process is called *garbage collection* and imposes additional overhead and complexity on the system.

ARIES

Algorithm for Recovery and Isolation Exploiting Semantics (ARIES)

Last Updated : 27 Aug, 2019

Algorithm for Recovery and Isolation Exploiting Semantics (ARIES) is based on the Write Ahead Log (WAL) protocol. Every update operation writes a log record which is one of the following :

1. **Undo-only log record:**
   Only the before image is logged. Thus, an undo operation can be done to retrieve the old data.
2. **Redo-only log record:**
   Only the after image is logged. Thus, a redo operation can be attempted.
3. **Undo-redo log record:**
   Both before images and after images are logged.

In it, every log record is assigned a unique and monotonically increasing log sequence number (LSN). Every data page has a page LSN field that is set to the LSN of the log record corresponding to the last update on the page. WAL requires that the log record corresponding to an update make it to stable storage before the data page corresponding to that update is written to disk. For performance reasons, each log write is not immediately forced to disk. A log tail is maintained in main memory to buffer log writes. The log tail is flushed to disk when it gets full. A transaction cannot be declared committed until the commit log record makes it to disk.

Once in a while the recovery subsystem writes a checkpoint record to the log. The checkpoint record contains the transaction table and the dirty page table. A master log record is maintained separately, in stable storage, to store the LSN of the latest checkpoint record that made it to disk. On restart, the recovery subsystem reads the master log record to find the checkpoint's LSN, reads the checkpoint record, and starts recovery from there on.

Key Features of Aries Recovery Algorithm in DBMS

1. **Write-Ahead Logging (WAL)**: This make sure that all changes are logged before they are applied to the database.
2. **Checkpointing**: To create a stable point in the database from which recovery can start.
3. **Three Phases of Recovery**: To ensure database recovery Analysis, Redo, and Undo phases are crucial.

Aries Recovery Algorithm Phases

1. Analysis Phase

The Analysis Phase identifies the database's state and determines which log entries need to be processed for recovery.

Example: Suppose we have a log with the following entries:

1. **<T1, Start>**
2. **<T1, Write(A, 10)>**
3. **<T1, Commit>**
4. **<T2, Start>**
5. **<T2, Write(B, 20)>**
6. **<T2, Abort>**

During the Analysis Phase:

- The system reads the log to identify active transactions (T2) and committed transactions (T1).
- It also records the state of database pages that were modified by active transactions.

Steps:

1. Scan the log to identify transactions and their states.
2. Create a list of dirty pages (pages that have been modified but not yet written to disk).

2. Redo Phase

The Redo Phase re-applies changes recorded in the log to ensure that all committed transactions are properly reflected in the database.

Example: Using the log from the previous example, the Redo Phase will:

- Reapply the changes made by T1 and T2. Since T1 committed, its changes to page A will be redone.
- T2's changes are ignored because the transaction was aborted.

Steps:

1. Replay every action taken during a committed transaction.
2. Ensure all modifications are applied to the database pages as per the log.

Code Example: Suppose we are applying redo for a write operation:

```
public void redoWrite(String pageId, String data) {
    // Fetch the page from disk
    Page page = fetchPageFromDisk(pageId);
    // Apply the changes from the log
    page.write(data);
    // Write the page back to disk
    writePageToDisk(page);
}
```

3. Undo Phase

The Undo Phase reverses changes made by aborted transactions, restoring the database to a consistent state by undoing any modifications from these transactions.

Example: From the log, we need to undo changes made by T2.

Steps:

1. Identify all operations from aborted transactions.
2. Apply undo operations to revert changes made by these transactions.

Code Example: Undoing a write operation might look like:

```
public void undoWrite(String pageId, String data) {
    // Fetch the page from disk
    Page page = fetchPageFromDisk(pageId);
    // Revert the changes from the log
    page.revert(data);
    // Write the page back to disk
    writePageToDisk(page);
}
```

Example of Aries Recovery Algorithm in DBMS

Let's consider a more detailed scenario:

Initial State:

- Page A: 0
- Page B: 0

Transactions:

- **T1** writes A = 10 and commits.
- **T2** writes B = 20 and aborts.

Log Entries

- **<T1, Start>**
- **<T1, Write(A, 10)>**
- **<T1, Commit>**
- **<T2, Start>**
- **<T2, Write(B, 20)>**
- **<T2, Abort>**

After Crash:

- Checkpoint indicates page A was modified.
- Analysis Phase finds T1 committed, T2 aborted.
- Redo Phase reapplies T1's write to page A.
- Undo Phase rolls back T2's write to page B.

Advantages of Aries Recovery Algorithm in DBMS

1. Fine-Grained Locking

ARIES uses fine-grained locking, allowing multiple transactions to access different parts of the same data item at the same time. This increases database concurrency, leading to better resource use and improved performance.

2. Efficient Crash Recovery

ARIES handles crash recovery efficiently through three phases: Analysis, Redo, and Undo:

- **Analysis**: Finds the checkpoint where recovery should start and identifies the transactions that were active during the crash.
- **Redo**: Reapplies all updates from the log to ensure the database reflects all committed transactions, even if they were not saved to disk before the crash.
- **Undo**: Reverts changes from incomplete transactions at the time of the crash to keep the database consistent.